

Homework2 part1 report

Name: Zhang Xianglong

SID: 1155241554

1. Experiment Result

Before introducing the overall structure of the project in detail, let's first present the score verification results of the 5 CVs provided on BlackBoard, which are consistent with the given groundtruth.

```
(py312) cjsmt:CV_Veri zhangxianglong$ python3 test.py
=== Running CV verification pipeline on test cases ===
[1/5] Processing /Users/zhangxianglong/CUHK/FTEC5660/Assignment/FTEC5660/homeworks/hw2/CV_Veri/datasets/CV_1.pdf ...
      average_score = 0.7667
[2/5] Processing /Users/zhangxianglong/CUHK/FTEC5660/Assignment/FTEC5660/homeworks/hw2/CV_Veri/datasets/CV_2.pdf ...
      average_score = 0.7333
[3/5] Processing /Users/zhangxianglong/CUHK/FTEC5660/Assignment/FTEC5660/homeworks/hw2/CV_Veri/datasets/CV_3.pdf ...
      average_score = 0.8333
[4/5] Processing /Users/zhangxianglong/CUHK/FTEC5660/Assignment/FTEC5660/homeworks/hw2/CV_Veri/datasets/CV_4.pdf ...
      average_score = 0.0
[5/5] Processing /Users/zhangxianglong/CUHK/FTEC5660/Assignment/FTEC5660/homeworks/hw2/CV_Veri/datasets/CV_5.pdf ...
      average_score = 0.0667

=== Evaluating against groundtruth ===
Scores:      [0.7667, 0.7333, 0.8333, 0.0, 0.0667]
Decisions:   [1, 1, 1, 0, 0]
Groundtruth: [1, 1, 1, 0, 0]
Correct: 5 / 5, final_score = 1.000

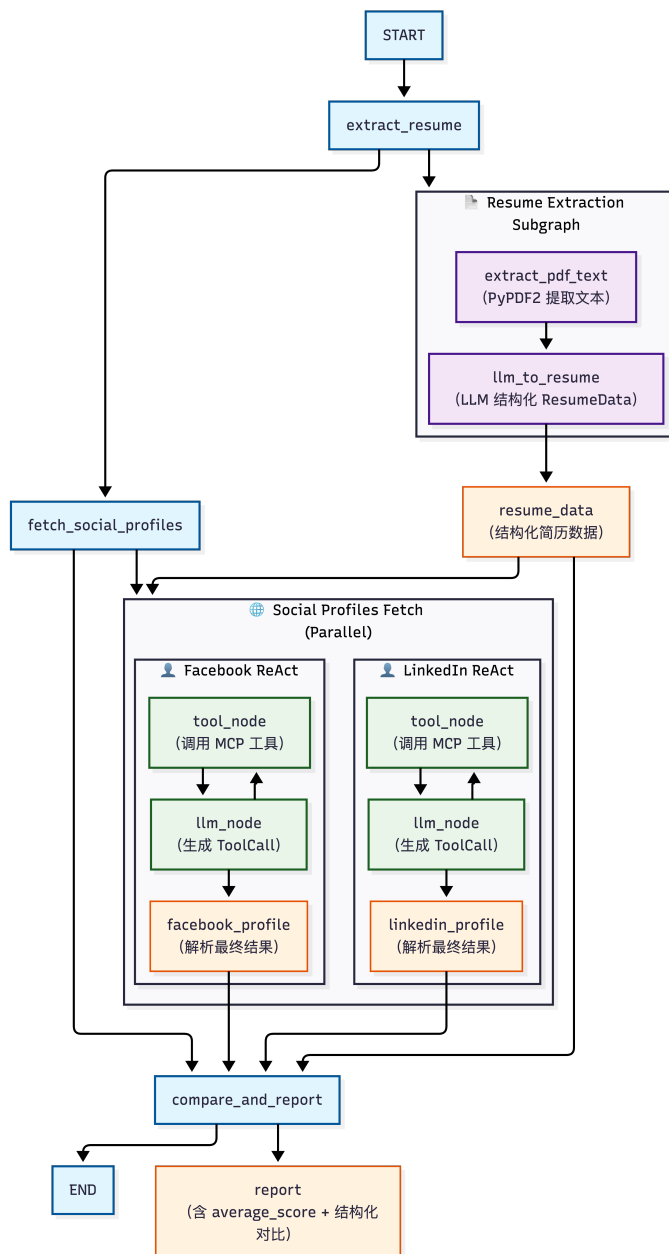
Report saved to: /Users/zhangxianglong/CUHK/FTEC5660/Assignment/FTEC5660/homeworks/hw2/CV_Veri/output/20260222_171429.json
(py312) cjsmt:CV_Veri zhangxianglong$
```

2. System Design

2.1. Workflow

The overall architecture diagram or workflow is as follows. In the project, I used the **LangGraph** technical framework to modularly organize the Agent system. It is divided into a total of **3 sub-modules**, that is, 3 Agent sub-systems work collaboratively. They are respectively:

- **extract_resume**: Use an LLM to extract useful key information from the text in the CV's PDF.
- **fetch_social_profiles**: Based on the information extracted from the resume such as name, city, country, etc., use the MCP tool to obtain public profiles on LinkedIn and Facebook. In this step, two LLMs are used in parallel to obtain public social media profiles from LinkedIn and Facebook.
- **compare_and_report**: Use an LLM to compare and score the resume content identified from the PDF obtained in the previous steps with the key information of the public social media profiles, and generate a comparison report.



2.2. State Definition

State is an important concept in the LangGraph framework, referring to the data state maintained between different agents in the entire graph. In this project, **State** is mainly used to store the structured output results of each stage of the workflow. Additionally, as the three stages described in the previous section are executed sequentially, the output results of the previous stage (such as `extract_resume` or `fetch_social_profiles`) serve as the input information source for the next stage (such as `compare_and_report`). The overall state definition is as follows (from [models.py](#)):

```

class CVState(TypedDict, total=False):
    """State structure used in LangGraph"""
    resume_path: str
    resume_data: Optional[ResumeData]
    linkedin_profile: Optional[LinkedInProfile]
    facebook_profile: Optional[FacebookProfile]
    report: Optional[VerificationReport]

```

2.3. Extract Resume

As the first part of the entire Agent system, the main task of this module is to read the content of the PDF using the PyPDF2 package based on the **resume_path** in CVState, and then extract key resume information, which is the **resume_data** part in CVState, through the method of LLM + prompt engineering. The core code is as follows(from [resume_extract_subgraph.py](#)):

```

class ResumeExtractState(TypedDict, total=False):
    resume_path: str
    ocr_text: str
    resume_data: Optional[ResumeData]

def build_resume_extract_graph(llm: Any, system_prompt: str):
    """
    Build the resume-extraction subgraph: extract_pdf_text → llm_to_resume_data → END
    llm: model used to turn the PDF text into structured JSON
    system_prompt: instructions that define the ResumeData JSON format
    """

    def node_extract_pdf_text(state: ResumeExtractState) -> ResumeExtractState:
        """Extract raw text from the PDF with PyPDF2 and store it in state.ocr_text."""
        ...

    def node_llm_to_resume_data(state: ResumeExtractState) -> ResumeExtractState:
        """Given PDF text and system_prompt, use the LLM to produce ResumeData JSON."""
        ...

    builder = StateGraph(ResumeExtractState)
    builder.add_node("extract_pdf_text", node_extract_pdf_text)
    builder.add_node("llm_to_resume", node_llm_to_resume_data)

    builder.add_edge(START, "extract_pdf_text")
    builder.add_edge("extract_pdf_text", "llm_to_resume")
    builder.add_edge("llm_to_resume", END)

    return builder.compile()

def node_extract_resume(state: CVState) -> CVState:
    """Node 1: run the resume-extraction subgraph (PDF text → LLM → ResumeData)."""
    resume_path = state["resume_path"]
    resume_prompt = load_prompt("resume_extract_system.txt")
    extract_app = build_resume_extract_graph(llm, resume_prompt)

```

```

state0: ResumeExtractState = {"resume_path": resume_path}
result = extract_app.invoke(state0)
state["resume_data"] = result.get("resume_data")
return state

```

2.4. Fetch Social Profiles

This part is the core focus of the entire project. In terms of implementation and design, it mainly uses the ReAct (Reasoning + Action) method, which repeatedly calls tools until the final result is obtained. Since LangGraph's native `create_react_agent` interface only supports synchronous calls to external tools, which conflicts with the asynchronous logic of the MCP ecosystem, I manually implemented `react_agent` (from [react_agent_subgraph.py](#)) in the project as the subgraph architecture of the `Fetch_Social_Profiles` module.

Both LinkedIn and Facebook require at least two MCP tool calls. (For example: To obtain a profile on LinkedIn, you first need to call `search_linkedin_people` to get the candidate's "person_id", which can then be used as a parameter to call the `get_linkedin_profile` tool to obtain complete personal profile information.) To obtain candidate profiles that are as accurate as possible and relevant to the resume content, in addition to the name, I also passed "location" as a parameter to `react_agent`. The specific code is as follows (from [graph.py](#)):

```

def node_fetch_social_profiles(state: CVState) -> CVState:
    """Node 2: run LinkedIn and Facebook LLM agents in parallel."""
    resume_data = state.get("resume_data")
    if not resume_data or not resume_data.get("name"):
        return state

    name = resume_data["name"]
    city = resume_data.get("city") or ""
    country = resume_data.get("country") or ""
    system_prompt_li = load_prompt("linkedin_agent_system.txt")
    system_prompt_fb = load_prompt("facebook_agent_system.txt")
    user_prompt_li = (
        "Please use the following resume information to search for and retrieve the"
        "candidate's LinkedIn profile.\n\n"
        f"Name: {name}\n"
        f"City: {city or 'not provided'}\n"
        f"Country: {country or 'not provided'}\n"
    )
    user_prompt_fb = ...

```

Since the acquisition of profile information from the two social media platforms, LinkedIn and Facebook, is independent of each other, the two Agents can be processed in parallel, with each calling its own tools to obtain profile information.

```

linkedin_profile = None
facebook_profile = None

with ThreadPoolExecutor(max_workers=2) as executor:
    future_li = executor.submit(asyncio.run, run_agent_once(linkedin_tools,
system_prompt_li, user_prompt_li))

```

```

future_fb = executor.submit(asyncio.run, run_agent_once(facebook_tools,
system_prompt_fb, user_prompt_fb))

for future in as_completed([future_li, future_fb]):
    msgs = future.result()
    is_linkedin = (future is future_li)
    platform = "LinkedIn" if is_linkedin else "Facebook"

    profile_data = None

    last_ai = next(
        (m for m in reversed(msgs) if isinstance(m, AIMessage)), None
    )

    if last_ai and not getattr(last_ai, "tool_calls", None) and last_ai.content:
        # Agent has finished; extract JSON from the text
        raw_str = extract_json_str_from_content(last_ai.content)
        data = extract_json_from_text(raw_str) if raw_str else None
        if data and not data.get("found") is False and not data.get("error"):
            profile_data = data
state["linkedin_profile"] = linkedin_profile
state["facebook_profile"] = facebook_profile
return state

```

2.5. Compare and Report

Finally, there is the Agent for comparison and report generation, which is also implemented using the method of LLM + prompt engineering. The resume information from the PDF and the public social media profile data are passed into the system_prompt. Then, scoring is performed based on the degree of matching, mainly scoring from three dimensions in the resume: **skills**, **educational background**, and **work experience**. The scoring rules are as follows (from [compare_and_report_system.txt](#)), and finally, the average of the three dimensions is taken as the comprehensive scoring result.

Notes:

- ****The score field reflects how reliable and consistent the provided resume information appears**:**
 - 0.0–0.3: very unreliable or completely mismatched (profiles differ greatly from the resume; likely a different person)
 - 0.4–0.6: partially reliable or partially matched (some consistency, but with significant differences)
 - 0.7–0.9: highly reliable or highly matched (mostly consistent, with only minor differences)
 - 1.0: fully reliable or perfectly matched (profiles and resume are fully consistent)
- ****When computing the overall score, you should consider**:**
 - Skill matching (weight: 0.3)
 - Work experience matching (weight: 0.4)
 - Education matching (weight: 0.2)
 - Other information matching (name, location, etc., weight: 0.1)